

TP — Construire et consommer une API REST

Licence 3 Informatique · Architecture REST · 65 min en séance + travail personnel

Objectifs

- Observer une requête et une réponse HTTP, et interpréter codes de statut et en-têtes.
- Consommer une API REST avec `fetch` en gérant les cas d'erreur.
- Implémenter une ressource REST avec CodeIgniter 4 en respectant un contrat d'interface.

Mise en place

Faites cette préparation avant la séance si possible. Il vous faut PHP, un projet CodeIgniter 4 récent (4.5 ou plus) et l'archive **TP_REST_Ci4_squelette.zip**.

1. Décompressez l'archive à la racine de votre projet : elle ajoute des fichiers dans `app/` et `public/` sans en écraser aucun.
2. Créez ou complétez le fichier `.env` à la racine du projet :

```
CI_ENVIRONMENT = development
app.baseURL = 'http://localhost:8080/'
app.indexPage = ''

# SQLite pour aller vite ; tout autre pilote convient (Postgre, SQLSRV...)
database.default.DBDriver = SQLite3
database.default.database = biblio.db
```

3. Ajoutez les deux routes de `app/Config/Routes_extrait.php` à la fin de `app/Config/Routes.php`.
4. Créez la table et les données de test : `php spark migrate`, puis `php spark db:seed LivreSeeder`.
5. Lancez le serveur de développement : `php spark serve`, puis ouvrez `http://localhost:8080/`.

La ligne `app.indexPage = ''` évite que `index.php` apparaisse dans les URL générées, en particulier dans l'en-tête `Location`.

Le contrat de l'API

Votre API devra respecter ce contrat. Il est imposé, comme dans un vrai projet où d'autres équipes développent les clients en parallèle. Les erreurs utilisent le format produit par le `ResponseTrait` de CodeIgniter : `{"status": 400, "error": 400, "messages": { ... }}`.

Requête	Corps envoyé	Réponse attendue
GET <code>/api/livres</code>	—	200 · <code>{"donnees": [...]}</code>
GET <code>/api/livres/{id}</code>	—	200 · le livre, ou 404
POST <code>/api/livres</code>	livre en JSON	201 · en-tête <code>Location</code> · le livre créé, ou 400

Requête	Corps envoyé	Réponse attendue
PUT /api/livres/{id}	livre complet	200 · le livre, ou 400 / 404
PATCH /api/livres/{id}	champs modifiés	200 · le livre, ou 400 / 404
DELETE /api/livres/{id}	—	204 (ou 200 justifié), ou 404

Un livre possède trois champs modifiables : `titre` (obligatoire), `auteur` (obligatoire) et `annee` (entier facultatif). Les règles de validation sont déjà écrites dans `app/Models/LivreModel.php`.

Niveau 1 — Observer (10 min)

JSONPlaceholder est une API publique de démonstration. Exécutez les commandes suivantes et observez la **totalité** de chaque réponse, en-têtes compris (option `-i`).

```
curl -i https://jsonplaceholder.typicode.com/posts/1
curl -i https://jsonplaceholder.typicode.com/posts/9999
curl -i -X POST https://jsonplaceholder.typicode.com/posts \
  -H "Content-Type: application/json" \
  -d '{"title": "Test", "body": "Bonjour", "userId": 1}'
curl -i https://jsonplaceholder.typicode.com/posts/101
curl -i -X DELETE https://jsonplaceholder.typicode.com/posts/1
```

Sous Windows, utilisez `curl.exe` dans PowerShell (et non l'alias `curl`), ou un client graphique comme Bruno ou Postman.

- Q1.** Pour la première requête, relevez le code de statut et la valeur de `Content-Type`.
- Q2.** Quel code obtenez-vous pour `/posts/9999` ? Qui, du client ou du serveur, doit corriger quelque chose ?
- Q3.** Après le POST : quel code ? Quel identifiant a été attribué ? L'en-tête `Location` est-il présent ?
- Q4.** Que renvoie `GET /posts/101` juste après ? Expliquez ce que cela révèle sur JSONPlaceholder.
- Q5.** Quel code renvoie le DELETE ? D'après le cours, quel autre code aurait-on pu attendre ?

Niveau 2 — Consommer (25 min)

Ouvrez `public/niveau2/app.js` et complétez les TODO 1 à 6, puis testez sur `http://localhost:8080/niveau2/`. Le client doit lister cinq articles, en publier un nouveau et en supprimer un.

Points d'attention

`fetch` ne lève pas d'exception sur un 404 ou un 500 : testez toujours `reponse.ok` ou `reponse.status`.

Pour envoyer du JSON, il faut à la fois `JSON.stringify(...)` et l'en-tête `Content-Type: application/json`.

Ouvrez l'onglet Réseau des outils de développement pendant vos tests : chaque clic doit y apparaître.

- Q6.** Après un POST réussi, l'article apparaît-il dans la liste rechargée ? Pourquoi ?

Q7. Votre page est servie par `localhost:8080` et appelle un autre domaine. Pourquoi le navigateur l'autorise-t-il ici ? Cherchez l'en-tête qui l'explique dans l'onglet Réseau.

Niveau 3 — Implémenter (30 min et plus)

Étape A : un endpoint écrit à la main (10 min)

Complétez `app/Controllers/Api/LivresManuel.php`, qui répond à `GET /api/manuel/livres/{id}` sans l'aide de `ResourceController`. Vous choisissez vous-même chaque code de statut.

```
curl -i http://localhost:8080/api/manuel/livres/1
curl -i http://localhost:8080/api/manuel/livres/99
```

Étape B : la ressource complète (20 min)

1. Lancez `php spark routes` et comparez la table affichée avec le contrat : quelle méthode du contrôleur répond à quel verbe ?
2. Complétez les méthodes de `app/Controllers/Api/Livres.php` dans cet ordre : `show`, `index`, `create`, `delete`, `update`.
3. Testez chaque méthode avec `curl` au fur et à mesure (commandes ci-dessous).
4. Recette finale : ouvrez `http://localhost:8080/client/`. Ce client est fourni et **ne doit pas être modifié** : s'il fonctionne, votre API respecte le contrat.

```
B=http://localhost:8080/api/livres
J="Content-Type: application/json"

curl -i $B
curl -i -X POST $B -H "$J" \
  -d '{"titre": "Design Patterns", "auteur": "Gamma et al.", "annee": 1994}'
curl -i -X POST $B -H "$J" -d '{"titre": ""}'
curl -i -X PATCH $B/4 -H "$J" -d '{"annee": 1995}'
curl -i -X PUT $B/4 -H "$J" -d '{"titre": "Design Patterns"}'
curl -i -X DELETE $B/4
```

Ces commandes utilisent la syntaxe `bash` (Linux, macOS, Git Bash sous Windows).

Trois pièges à connaître

Filtre CSRF. S'il est activé dans les filtres globaux de `app/Config/Filters.php`, il bloque `POST`, `PUT`, `PATCH` et `DELETE`. Excluez les routes `api/*`.

Corps JSON. `getPost()` reste vide quand le corps est en JSON : utilisez `$this->request->getJSON(true)`.

CORS. Un client servi depuis une autre origine est bloqué par le navigateur. Ici, les clients sont dans `public/` : même origine, pas de problème. Sinon, voir le filtre `cors` et `app/Config/Cors.php` (CodeIgniter 4.5+).

Q8. `respondCreated()` renvoie-t-il 201 ? Ajoute-t-il l'en-tête `Location` ? Vérifiez avec `curl -i`.

Q9. Pour `DELETE`, avez-vous choisi 200 ou 204 ? Justifiez en une phrase, en lien avec le corps de la réponse.

Q10. Un `PUT` qui omet le champ `auteur` doit-il réussir ? Et un `PATCH` qui omet ce même champ ? Montrez comment votre code distingue les deux.

Bonus

- **Pagination.** `GET /api/livres?page=2&par_page=10`, avec les clés `page`, `par_page` et `total` dans la réponse.
- **Versionnement.** Servir la ressource sous `/api/v1/livres` à l'aide d'un groupe de routes.
- **Emprunts.** Implémenter la ressource `emprunts` conçue en binôme : `POST /api/emprunts` renvoie 409 si le livre est déjà emprunté, et le retour d'un livre se fait par `PATCH`.

À rendre

- Votre projet, sans les dossiers `vendor/` et `writable/`.
- Un fichier `REPONSES.md` contenant le tableau de conception de votre binôme et les réponses aux questions Q1 à Q10.